Floating-Point MAC Unit: Design and Implementation

Prepared By:

Group Number: 43

Sarim Zafeer Farooqui
Gokul L P
Sivaganesh Locharla
Muhammad Anas A K

November 29, 2024

Abstract

Multiply-Accumulate (MAC) operations form the cornerstone of modern digital signal processing and are particularly critical in Artificial Neural Network (ANN) accelerators, where they dominate computational workloads. This report details the design and implementation of a high-performance 32-bit floating-point MAC unit based on the IEEE 754 single-precision format. The MAC unit integrates three key functional blocks: a tree-based multiplier for efficient and parallelized mantissa multiplication, an adder for accumulation, and normalization logic to ensure precision and compliance with the IEEE 754 standard.

To optimize performance, the design leverages architectural techniques aimed at reducing the critical path delay, minimizing area, and ensuring power efficiency. Extensive simulations were conducted to validate the functional correctness of the MAC unit, covering a variety of input scenarios, including positive, negative, and fractional operands. Furthermore, the physical design process—encompassing synthesis, floorplanning, placement, clock tree synthesis, routing, and layout verification—was completed, with an emphasis on achieving zero DRC errors and successful LVS checks.

This document provides a comprehensive analysis of the theoretical concepts underpinning the design, the step-by-step implementation methodology, and the performance evaluation metrics. The MAC unit is benchmarked against key performance indicators, including latency, power consumption, and area utilization, making it a robust candidate for integration into ANN accelerators and other high-speed digital processing applications.

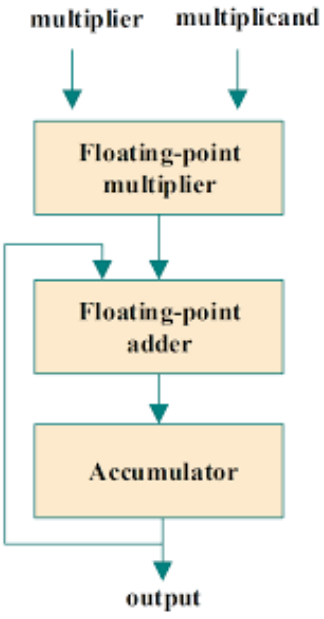

Figure 1: Block Diagram of the 32-bit Floating-Point MAC Unit

Contents

1 Introduction **3**

2 Floating-Point Arithmetic **3**
2.1 IEEE 754 Single-Precision Standard . . . 3
2.2 Floating-Point Operations . . . 4
2.2.1 Addition and Subtraction . . . 4
2.2.2 Multiplication . . . 4
2.3 Advantages of Floating-Point Arithmetic . . . 5
2.4 Relevance to the MAC Unit . . . 5

3 MAC Unit Architecture **5**
3.1 Overview of the MAC Operation . . . 6
3.2 Components of the MAC Unit . . . 6
3.3 Tree-Based Multiplier . . . 6
3.4 Floating-Point Adder . . . 7
3.5 Accumulator . . . 8
3.6 Integration of Components . . . 8
3.7 Relevance to Neural Networks . . . 9
3.8 Benefits of the Designed MAC Unit . . . 9

4 Implementation Methodology **10**
4.1 Verilog Design . . . 10
4.1.1 Module Structure . . . 10
4.2 Major Module Functions and Support Functions . . . 11
4.3 Major Module Functions . . . 11
4.4 Support Functions . . . 12
4.5 +4 Error and Its Solution . . . 12
4.5.1 Parameterization and Modularity . . . 22
4.6 Testbench Design . . . 22
4.6.1 Testbench Structure . . . 22
4.6.2 Edge Case Considerations . . . 24
4.7 Simulation and Verification . . . 24
4.8 Synthesis and Post-Synthesis Verification . . . 25

5 Simulation Results **25**
5.1 Pre-Synthesis Verification . . . 26
5.1.1 Testbenches and Input Generation . . . 26
5.1.2 Simulation Results and Waveforms . . . 26
5.2 Post-Synthesis Verification . . . 27
5.2.1 Simulation Results and Post-Synthesis Waveforms . . . 27
5.3 Performance Metrics . . . 28
5.3.1 Standard Cell Usage . . . 28
5.3.2 Power Consumption . . . 29
5.3.3 Chip Area . . . 29
5.3.4 Skew and Slack Analysis . . . 29

6 Physical Design **30**
6.1 Floorplan . . . 30
6.1.1 Objective . . . 30
6.1.2 Process . . . 30
6.1.3 Results . . . 31
6.2 Placement . . . 31
6.2.1 Objective . . . 31
6.2.2 Process . . . 32
6.2.3 Results . . . 32
6.3 Clock Tree Synthesis (CTS) . . . 32
6.3.1 Objective . . . 33
6.3.2 Process . . . 33
6.3.3 Results . . . 33
6.4 Routing . . . 34
6.4.1 Objective . . . 34
6.4.2 Process . . . 34
6.4.3 Results . . . 35
6.5 Final Layout (GDS) . . . 35
6.5.1 Results . . . 35
6.6 Post Physical Design RTL Verification . . . 36
6.6.1 Objective . . . 36
6.6.2 Methodology . . . 36
6.6.3 Results . . . 37
6.7 LVS Report . . . 37
6.7.1 Results . . . 37
6.8 DRC Report . . . 37
6.8.1 Results . . . 38
6.9 STA Report and Power Consumption . . . 38
6.9.1 Results . . . 38

7 Group Work Division **38**

8 Conclusion **40**

1 Introduction

The Multiply-Accumulate (MAC) unit is a critical element in modern digital computation, widely used in Digital Signal Processing (DSP) and Artificial Neural Network (ANN) accelerators. The MAC operation involves a sequence of multiplications followed by additions, expressed as:

$$MAC = \sum_{i=1}^{N} (A_i \times B_i) + C \tag{1}$$

In ANN applications, MAC operations are performed billions of times per second to process weights and input data across network layers. The floating-point MAC unit provides flexibility and precision for applications requiring large dynamic ranges, such as scientific computation, image processing, and machine learning.

This report discusses the design and implementation of a 32-bit floating-point MAC unit using the IEEE 754 single-precision standard. The design is optimized for accuracy, speed, and area efficiency.

2 Floating-Point Arithmetic

Floating-point arithmetic is the foundation for designing the Multiply-Accumulate (MAC) unit in this project. Unlike fixed-point representation, floating-point numbers offer a dynamic range and precision, making them ideal for applications like scientific computing, signal processing, and neural networks. This section explores the IEEE 754 floating-point standard, the nuances of floating-point operations, and their relevance to this project.

2.1 IEEE 754 Single-Precision Standard

The IEEE 754 standard is the universally accepted format for representing and manipulating floating-point numbers in digital systems. It ensures compatibility and precision across diverse hardware and software platforms. In the single-precision format, each number is represented using 32 bits, divided into three fields:

- **Sign (1 bit):** Indicates the sign of the number. A value of 0 represents a positive number, while 1 denotes a negative number.
- **Exponent (8 bits):** Encodes the power of two to which the mantissa is multiplied. The exponent is stored in a biased form, where a bias of 127 is subtracted to retrieve the actual exponent.
- **Mantissa (23 bits):** Represents the significant digits of the number. An implicit leading 1 is assumed for normalized numbers, effectively providing a 24-bit mantissa.

The floating-point number is mathematically expressed as:

$$Z = (-1)^S \times 2^{(E-127)} \times (1.M) \tag{2}$$

Where S is the sign bit, E is the exponent, and M is the mantissa.

This representation enables the encoding of both extremely large and small numbers, overcoming the limitations of fixed-point arithmetic.

2.2 Floating-Point Operations

Floating-point arithmetic involves the separate handling of the sign, exponent, and mantissa fields. The main operations—addition, subtraction, multiplication, and division—are executed through a series of well-defined steps.

2.2.1 Addition and Subtraction

Addition and subtraction of floating-point numbers require aligning the exponents of the operands:

1. **Exponent Alignment:** The smaller exponent is incremented while the corresponding mantissa is shifted right, ensuring both numbers have the same exponent.
2. **Mantissa Arithmetic:** Once aligned, the mantissas are added or subtracted based on the signs of the operands.
3. **Normalization:** The result is normalized by adjusting the exponent and mantissa to conform to the IEEE 754 standard.

For example, consider adding two floating-point numbers 3.75 (1.111×2^1) and 0.875 (1.110×2^0):

- Align the exponents: $3.75 = 1.111 \times 2^1$ and $0.875 = 0.111 \times 2^1$.
- Add the mantissas: $1.111 + 0.111 = 10.000$.
- Normalize: $10.000 \times 2^1 = 1.000 \times 2^2 = 4.00$.

2.2.2 Multiplication

Floating-point multiplication involves:

1. **Sign Calculation:** The XOR of the operand signs determines the sign of the result.
2. **Exponent Addition:** The exponents of the operands are added, and the bias (127) is subtracted to account for the double biasing.
3. **Mantissa Multiplication:** The mantissas, including their implicit leading 1s, are multiplied. The result may require normalization if it exceeds the range $[1.0, 2.0)$.

For instance, multiplying 1.5 (1.100×2^0) and 0.5 (1.000×2^{-1}):

- Calculate the sign: $0 \oplus 0 = 0$ (positive).
- Add the exponents: $0 + (-1) - 127 = -128 + 127 = -1$.
- Multiply the mantissas: $1.100 \times 1.000 = 1.100$.
- Normalize: $1.100 \times 2^{-1} = 0.75$.

2.3 Advantages of Floating-Point Arithmetic

Floating-point arithmetic offers significant advantages over fixed-point representation:

- **Dynamic Range:** The use of exponents allows representation of both very large and very small numbers.
- **Precision:** Higher precision is achievable through normalization, ensuring minimal loss of significant digits.
- **Standardization:** The IEEE 754 format ensures compatibility across systems, making it suitable for portable computations.

2.4 Relevance to the MAC Unit

The floating-point MAC unit combines multiplication and accumulation operations, fundamental in tasks like convolution in neural networks. By leveraging floating-point arithmetic, the MAC unit achieves:

- **High Precision:** Essential for iterative computations where errors can propagate.
- **Dynamic Range:** Accommodates the diverse scales of weights and inputs in neural networks.
- **Efficiency:** Optimized multiplication and addition reduce computational delays and power consumption.

This theoretical foundation guided the design and implementation of the floating-point MAC unit, detailed in subsequent sections.

3 MAC Unit Architecture

The Multiply-Accumulate (MAC) unit is a critical building block in digital signal processing and neural network computations. It combines two fundamental arithmetic operations—multiplication and addition—into a single, efficient operation. This section elaborates on the design principles, key components, and theoretical basis for the architecture of the floating-point MAC unit.

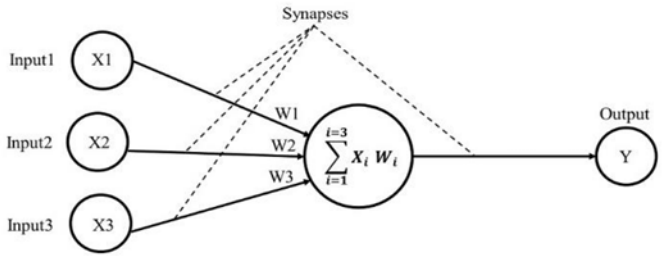

Figure 2: The architecture of a single node in NN, which demonstrates the core building unite is multiply accumulate

3.1 Overview of the MAC Operation

The MAC operation performs:

$$MAC = A \times B + Acc \tag{3}$$

where A and B are the operands, and Acc is the accumulator that stores the result of previous operations. This iterative operation is vital in many applications, such as:

- **Convolution in Neural Networks:** Used to compute weighted sums in convolutional and fully connected layers.
- **Digital Filters in DSP:** Implements Finite Impulse Response (FIR) and Infinite Impulse Response (IIR) filters.
- **Matrix Multiplications:** Essential for linear algebra operations in machine learning and scientific computing.

The floating-point MAC unit enhances these operations by supporting a dynamic range and high precision, making it suitable for diverse applications.

3.2 Components of the MAC Unit

The MAC unit comprises three main components:

1. **Multiplier:** Computes the product of the input operands.
2. **Adder:** Adds the product to the accumulator's stored value.
3. **Accumulator:** Retains intermediate results for iterative operations.

Each component is designed to handle floating-point arithmetic, ensuring precision and efficiency.

3.3 Tree-Based Multiplier

The multiplier is the most computationally intensive component of the MAC unit. It calculates the product of the mantissas of the input operands, adjusts the exponents, and determines the result's sign. A tree-based multiplier architecture was chosen to optimize performance:

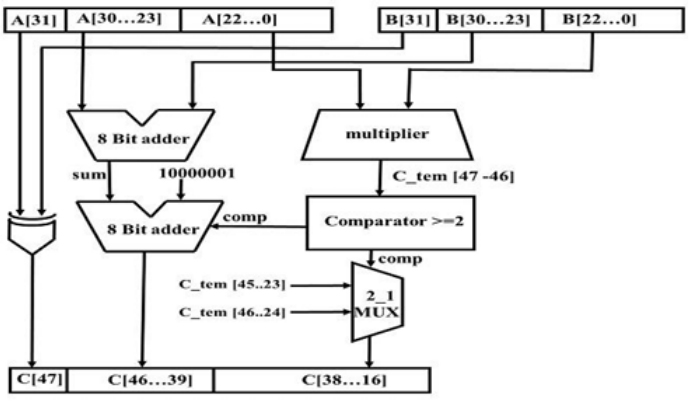

Figure 3: Block diagram of single-precision floating point multiplier

- **Partial Product Generation:** The mantissas of the two operands are multiplied, generating multiple partial products.
- **Reduction Using Binary Tree Adders:** The partial products are summed using a tree structure of adders, significantly reducing the propagation delay from $O(n)$ to $O(\log n)$, where n is the number of partial products.
- **Normalization:** Ensures the product lies within the range $[1.0, 2.0)$ by adjusting the exponent and shifting the mantissa as needed.

For example, consider multiplying $A = 1.5$ (1.100×2^0) and $B = 0.5$ (1.000×2^{-1}):

- Multiply mantissas: $1.100 \times 1.000 = 1.100$.
- Add exponents: $0 + (-1) = -1$.
- Normalize: The product 1.100×2^{-1} remains normalized, and the result is 0.75.

This approach minimizes hardware complexity and power consumption while maintaining high throughput.

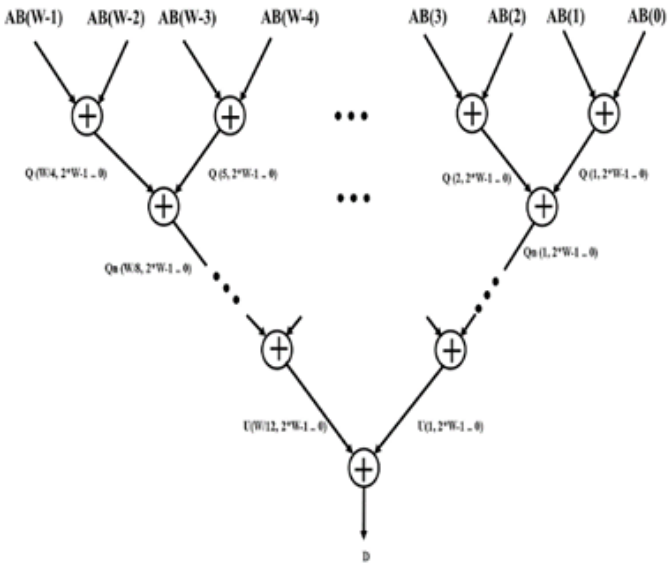

Figure 4: Block diagram of proposed parametrized multiplication build using tree of adders

3.4 Floating-Point Adder

The adder is responsible for combining the multiplication result with the value stored in the accumulator. Floating-point addition involves:

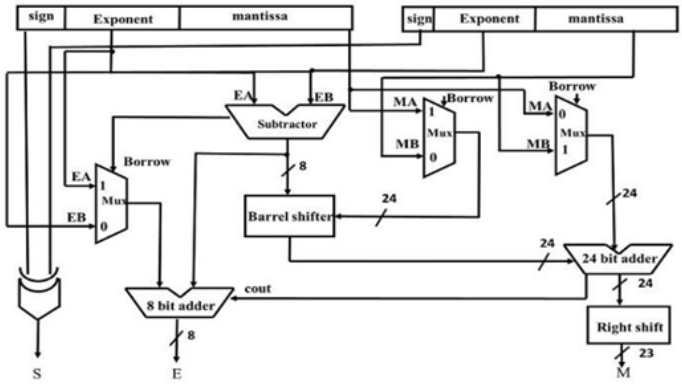

Figure 5: Block diagram of floating point addition

1. **Exponent Alignment:** The smaller exponent is incremented, and the corresponding mantissa is shifted right to align the exponents.
2. **Mantissa Addition:** Adds or subtracts the mantissas based on their signs.
3. **Normalization and Rounding:** Adjusts the result to the IEEE 754 format, discarding any excess bits while preserving accuracy.

For instance, consider adding the results of two MAC iterations:

- First result: $A \times B = 2.5$ (1.010×2^1).
- Second result: $A \times B + C = 4.5$ (1.001×2^2).
- Alignment: Align exponents by shifting the smaller value's mantissa.
- Addition: $1.010 \times 2^1 + 1.000 \times 2^1 = 1.101 \times 2^2 = 4.5$.

3.5 Accumulator

The accumulator stores intermediate results of MAC operations, enabling iterative computations. It plays a crucial role in ANN accelerators, where multiple iterations of MAC operations are required to compute weighted sums. The accumulator's design ensures:

- **Low Latency:** Enables high-speed operations by minimizing data transfer delays.
- **Precision Preservation:** Prevents errors from accumulating over multiple iterations.
- **Efficient Reset Mechanism:** Supports clearing the stored value without interrupting the computation flow.

3.6 Integration of Components

The MAC unit integrates the multiplier, adder, and accumulator in a pipelined structure to enhance throughput:

- **Stage 1: Multiplication.** Partial products are computed and reduced.

- **Stage 2: Addition.** The multiplication result is added to the accumulator.
- **Stage 3: Normalization.** Ensures the output conforms to the IEEE 754 standard.

This pipelining ensures that new inputs can be processed even before previous outputs are fully computed, maximizing efficiency.

3.7 Relevance to Neural Networks

In ANNs, the MAC unit computes the weighted sums of inputs at each neuron. The weights and inputs are typically stored as floating-point numbers to handle a wide range of values. The MAC operation:

- **Computes Neuron Outputs:** Each neuron performs $MAC = \sum(Input \times Weight)$.
- **Handles Batch Processing:** The accumulator enables efficient processing of multiple inputs in a batch.
- **Supports Precision Requirements:** Floating-point representation ensures minimal loss of accuracy in deep networks.

For example, in a convolutional layer of a neural network, the MAC unit computes the dot product of the input feature map and the kernel weights for each pixel, enabling rapid and precise convolutional operations.

3.8 Benefits of the Designed MAC Unit

The floating-point MAC unit designed in this project offers:

- **High Speed:** The tree-based multiplier and pipelined structure optimize performance for real-time applications.
- **Low Area Utilization:** Efficient use of logic gates ensures minimal hardware overhead.
- **Energy Efficiency:** Reduced power consumption, critical for portable and embedded systems.
- **Precision and Scalability:** Support for floating-point operations ensures high accuracy and adaptability to larger networks.

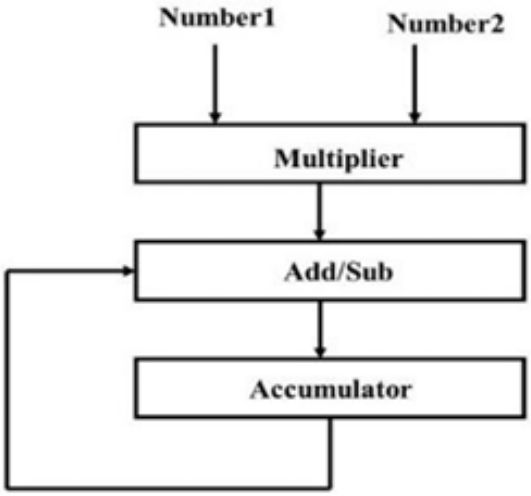

Figure 6: Block diagram of the MAC unit

4 Implementation Methodology

The design of the floating-point Multiply-Accumulate (MAC) unit is carried out in a systematic manner, combining theoretical principles with practical hardware implementation. This section outlines the methodology used to implement the MAC unit, focusing on the Verilog design, the creation of a functional testbench, and the simulation process. We also discuss the steps taken for synthesis, post-synthesis verification, and performance evaluation.

4.1 Verilog Design

Verilog is a hardware description language (HDL) commonly used for modeling digital systems. The MAC unit was implemented using Verilog, adhering to best practices for synthesizable, modular, and parameterized code. The Verilog design allows the unit to be flexible, scalable, and easy to modify, making it suitable for integration into larger systems.

4.1.1 Module Structure

The MAC unit is designed as a single module, which performs both the multiplication and accumulation of the operands. The inputs are 32-bit floating-point numbers, and the output is also a 32-bit floating-point number. The module operates synchronously with a clock signal, which ensures that all computations are completed in a defined time frame.

The Verilog code for the MAC unit is broken down into the following key parts:

- **Inputs and Outputs:** The MAC unit has three main inputs: two 32-bit operands (A and B) and a clock signal (CLK). It also has a reset input (RST_N) to initialize the operation. The output is a 32-bit result (MAC_OUT) that stores the accumulated value.

- **Multiplier:** The multiplier section handles the multiplication of the two input operands, following the IEEE 754 floating-point multiplication procedure. It uses a tree-based structure to compute partial products and reduce them efficiently.

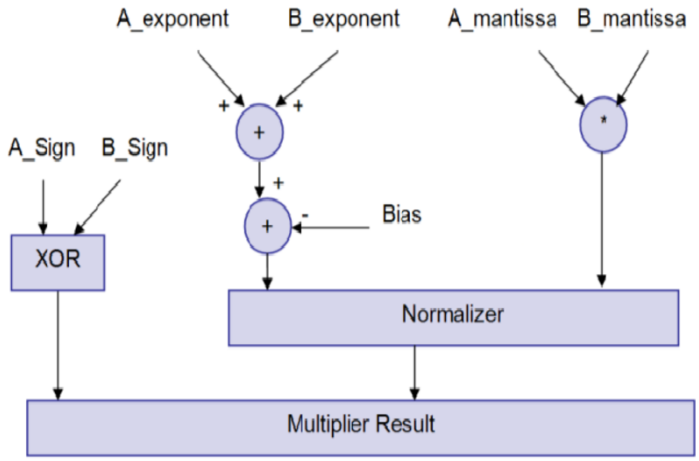

Figure 7: IEEE 754 floating-point multiplier

- **Adder:** The adder receives the product from the multiplier and adds it to the previous result stored in the accumulator.

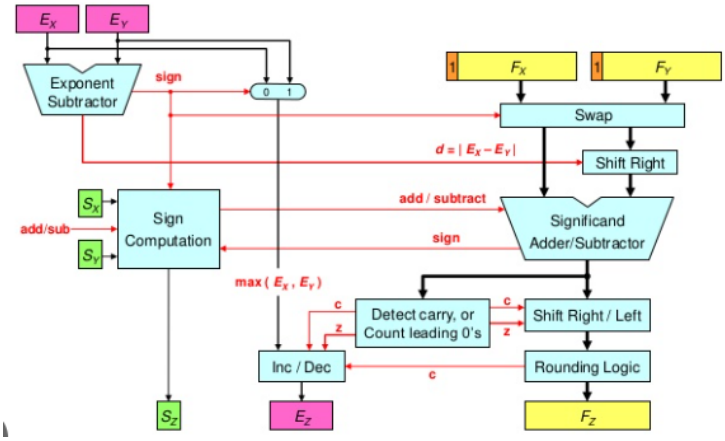

Figure 8: IEEE 754 floating-point adder

- **Accumulator:** The accumulator stores the result of the addition for the next operation, enabling iterative computations.

4.2 Major Module Functions and Support Functions

The design and implementation of the floating-point Multiply-Accumulate (MAC) unit involve several key modules, each with distinct responsibilities to ensure accurate and efficient operation. This section describes the major modules, their functionalities, and the supporting components that contribute to the overall design.

4.3 Major Module Functions

The MAC unit consists of the following primary modules:

- **Mac (Top-Level Module):** This is the top-level module that integrates the floating-point adder, floating-point multiplier, and a register to hold intermediate values. It ensures synchronous, clocked operation of the MAC unit.
- **Multiplier:** The floating-point multiplier unit performs multiplication of two input operands while normalizing the result to the IEEE 754 single-precision format. The process includes handling mantissa multiplication and adjusting the exponent.
- **Float_add:** This module handles floating-point addition. It sums the two input operands and normalizes the result to comply with the IEEE 754 single-precision format.
- **zeroDetect:** A specialized module designed to detect all-zero input conditions. If the input is zero, the module prevents the addition of the bias value, which could otherwise result in a +4 error in the final output.
- **TreeMultiplier:** Implements a tree-based multiplier architecture for fast and efficient multiplication. This approach reduces latency and improves performance, making it suitable for high-speed operations.
- **customAdder:** This module is a custom-designed adder for adding the 48-bit mantissa-based partial products. It is built using 16-bit Brent Kung Adders for optimized addition performance.

4.4 Support Functions

The design also incorporates several helper functions that play a crucial role in enabling the operation of the primary modules:

- **8-Bit Ripple Carry Adder:** Used for basic addition tasks within the design, ensuring low complexity and reliable performance.
- **Multiplexers:** Used for signal selection and routing, allowing flexible operation and data flow within the MAC unit.
- **16-Bit Brent Kung Adder:** A fast and efficient adder used within the customAdder module to sum partial products generated by the multiplier. The Brent Kung architecture ensures a logarithmic delay, improving overall speed.

4.5 +4 Error and Its Solution

The architecture provided in the reference paper includes a critical constraint regarding all-zero inputs, particularly for the exponent. If the exponent is all zero, adding the bias value of `10000001` for normalization would result in a constant offset of +4. This finite error cannot be corrected later in the computation, potentially leading to inaccurate results.

To address this issue, we developed an OR tree-based zeroDetect block that:

- Checks if all inputs are zero.
- Generates a signal to indicate the all-zero condition.

Based on the signal from the zeroDetect block:

- The bias adjustment step is bypassed to prevent the +4 error.
- This optimization ensures that the MAC unit operates correctly under clocked conditions, even when encountering all-zero inputs.

The zeroDetect block is a critical addition to the design, improving its reliability and precision, especially for real-time, clocked operations. This solution not only eliminates the error but also enhances the robustness of the overall MAC architecture.

The Verilog code for the MAC unit module is as follows:

Listing 1: Verilog Code for Floating-Point MAC Unit

```
module mac(In1, In2 , MacOut , CLK , rst);

    input [31:0] In1,In2;                                          //
       Inputs to MAC unit
    output [31:0] MacOut;                                          //
       Output of MAC unit
    input CLK,rst;                                                 //Clk
       and rst lines for MAC unit

    reg [31:0] In1_reg     = 32'b0;                                //
       Register to Hold In1 value
    reg [31:0] In2_reg     = 32'b0;                                //
       Register to Hold In2 value
    reg [31:0] accumulator = 32'b0;                                //
       Register to Hold Accumulator


    //Intermediate wires

    wire [31:0] mult_Out;
    wire [31:0] add_Out;
    wire [31:0] Out;

    //Top level Connections

    Multiplier M1(In1_reg,In2_reg,mult_Out);
    float_add F1(mult_Out,MacOut,add_Out);
    assign MacOut = accumulator;

    //Describing functionality of circuit with clock and reset

    always @(posedge CLK)
    begin
        //Clearing all registers
        if(~rst)
            begin
                In1_reg <= 0;
                In2_reg <= 0;
                accumulator<=0;
            end
        else
        //Moving data into input registers and data out to output
           registers
            begin
```

```
                In1_reg <= In1;
                In2_reg <= In2;
                accumulator<=add_Out;
        end
    end
endmodule


//Single Precision Multiplier Unit Implementation


module Multiplier(A,B,C);

    input [31:0] A,B;                                   //inputs to
       Multiplier A,B
    output [31:0] C;                                    //output of
       Multiplier C

    //Intermediate Values

    wire [47:0] X;
    wire [7:0] X_s;
    wire [7:0] exponent_out;
    wire [1:0] dummy_carry;
    wire zeroCheck;

    // Sign Bit Calculation

    assign C[31] = A[31]^B[31];

    // Mantissa Bits Calculations

    TreeMultiplier U1(A[22:0],B[22:0],X) ;
    assign comp_out = (X[47]==1'b1) ? 1'b1 : 1'b0 ;
    Mux um1(X[45:23],X[46:24],X[47],C[22:0]);

    // Exponent Bits Calculations

    ripple_carry_adder_8bit ua1(A[30:23],B[30:23],1'b0,X_s,dummy_carry
       [0]);
    ripple_carry_adder_8bit ua2(X_s,8'b10000001,X[47],exponent_out,
       dummy_carry[1]);
    Mux8bit um2(8'b0,exponent_out,zeroCheck,C[30:23]);
    zeroDetect Uz1 (A[30:23],B[30:23],zeroCheck);

endmodule


// Implementing single precision Float adder

module float_add (
    input  [31:0] a,
    input  [31:0] b,
    output [31:0] result
);


    wire sign_a = a[31];
```

```
    wire [7:0] exp_a = a[30:23];
    wire [23:0] mant_a = {1'b1, a[22:0]}; // Add implicit leading 1

    wire sign_b = b[31];
    wire [7:0] exp_b = b[30:23];
    wire [23:0] mant_b = {1'b1, b[22:0]}; // Add implicit leading 1

    // Compare exponents and align mantissas
    wire [7:0] exp_diff;
    wire [23:0] mant_a_shifted, mant_b_shifted;
    wire [7:0] exp_large;
    wire sign_large;

    assign exp_diff = (exp_a > exp_b) ? (exp_a - exp_b) : (exp_b -
       exp_a);
    assign exp_large = (exp_a > exp_b) ? exp_a : exp_b;
    assign sign_large = (exp_a > exp_b) ? sign_a : sign_b;

    // Shift the smaller mantissa to align with the larger exponent
    assign mant_a_shifted = (exp_a > exp_b) ? mant_a : (mant_a >>
       exp_diff);
    assign mant_b_shifted = (exp_b > exp_a) ? mant_b : (mant_b >>
       exp_diff);

    // Add or subtract mantissas based on the signs
    wire [24:0] mantissa_sum;
    wire [24:0] mantissa_diff;
    wire [24:0] mantissa_result;

    assign mantissa_sum = (sign_a == sign_b) ? (mant_a_shifted +
       mant_b_shifted) : 25'b0;
    assign mantissa_diff = (sign_a != sign_b) ?
        ((mant_a_shifted > mant_b_shifted) ? (mant_a_shifted -
           mant_b_shifted) : (mant_b_shifted - mant_a_shifted)) : 25'b0
           ;

    assign mantissa_result = (sign_a == sign_b) ? mantissa_sum :
       mantissa_diff;

    // Normalize result
    wire [24:0] normalized_mantissa;
    wire [7:0] normalized_exponent;
    wire normalized_sign;

    assign normalized_mantissa = (mantissa_result[24]) ?
       mantissa_result >> 1 : mantissa_result;
    assign normalized_exponent = (mantissa_result[24]) ? exp_large + 1
       : exp_large;


    wire [24:0] final_mantissa;
    wire [7:0] final_exponent;
    assign final_mantissa = (normalized_mantissa[23]) ?
       normalized_mantissa : normalized_mantissa << 1;
    assign final_exponent = (normalized_mantissa[23]) ?
       normalized_exponent : normalized_exponent - 1;

    // sign
```

```
    assign normalized_sign = (sign_a == sign_b) ? sign_a : (
       mant_a_shifted > mant_b_shifted ? sign_a : sign_b);

    //final result
    assign result = {normalized_sign, final_exponent, final_mantissa
       [22:0]};

endmodule



// Zero Detect Unit Implementation

module zeroDetect(A1,B1,status);

    input [7:0] A1,B1;                              //Input bits to module
    output status;                                  //Status bit indicating
        output

    wire [13:0] temp;

    assign temp[0] = A1[0] | B1[0];
    assign temp[1] = A1[1] | B1[1];
    assign temp[2] = A1[2] | B1[2];
    assign temp[3] = A1[3] | B1[3];
    assign temp[4] = A1[4] | B1[4];
    assign temp[5] = A1[5] | B1[5];
    assign temp[6] = A1[6] | B1[6];
    assign temp[7] = A1[7] | B1[7];
    assign temp[8] = temp[0] | temp[1];
    assign temp[9] = temp[2] | temp[3];
    assign temp[10] = temp[4] | temp[5];
    assign temp[11] = temp[6] | temp[7];
    assign temp[12] = temp[8] | temp[9];
    assign temp[13] = temp[10] | temp[11];

    assign status = temp[12] | temp[13];

endmodule


// 8-bit Ripple Carry Adder Module


module ripple_carry_adder_8bit (
    input [7:0] A,    // First 8-bit input
    input [7:0] B,    // Second 8-bit input
    input Cin,        // Carry input
    output [7:0] Sum, // 8-bit Sum output
    output Cout       // Final Carry output
);
    wire [7:0] carry; // Internal carry wires

    // Instantiate 8 Full Adders
    full_adder fa0 (A[0], B[0], Cin,    Sum[0], carry[0]);
    full_adder fa1 (A[1], B[1], carry[0], Sum[1], carry[1]);
    full_adder fa2 (A[2], B[2], carry[1], Sum[2], carry[2]);
    full_adder fa3 (A[3], B[3], carry[2], Sum[3], carry[3]);
```

```
    full_adder fa4 (A[4], B[4], carry[3], Sum[4], carry[4]);
    full_adder fa5 (A[5], B[5], carry[4], Sum[5], carry[5]);
    full_adder fa6 (A[6], B[6], carry[5], Sum[6], carry[6]);
    full_adder fa7 (A[7], B[7], carry[6], Sum[7], Cout);
endmodule


// 1 Bit Full adder Implementation

module full_adder (
    input a,          // First input bit
    input b,          // Second input bit
    input cin,        // Carry input
    output sum,       // Sum output
    output cout       // Carry output
);
    assign sum = a ^ b ^ cin;        // XOR for sum
    assign cout = (a & b) | (b & cin) | (a & cin); // Carry-out
endmodule



// Tree Multiplier implementation

module TreeMultiplier(A,B,D);


        input [22:0] A, B;                                  //Inputs to
           module
        output [47:0] D;                                    //Output to
           module

        //Intermediate Wires

        wire [23:0] A_m ;
        wire [23:0] B_m;
        wire [47:0] S_m  [11:0];
        wire [47:0] Q_m  [5:0];
        wire [47:0] Qn_m [2:0];
        wire [47:0] U ;
        wire [47:0] D ;
        wire [47:0] AB_m [23:0];

        //Appending the hidden bit
        assign A_m = {1'b1,A};
        assign B_m = {1'b1,B};

        //Generating Partial Products

        assign AB_m[0]= (B_m[0]==1'b1) ? ({24'b0,A_m}) : {48'b0} ;
        assign AB_m[1]= (B_m[1]==1'b1) ? ({23'b0,A_m,1'b0}) : {48'b0} ;
        assign AB_m[2]= (B_m[2]==1'b1) ? ({22'b0,A_m,2'b0}) : {48'b0} ;
        assign AB_m[3]= (B_m[3]==1'b1) ? ({21'b0,A_m,3'b0}) : {48'b0} ;
        assign AB_m[4]= (B_m[4]==1'b1) ? ({20'b0,A_m,4'b0}) : {48'b0} ;
        assign AB_m[5]= (B_m[5]==1'b1) ? ({19'b0,A_m,5'b0}) : {48'b0} ;
        assign AB_m[6]= (B_m[6]==1'b1) ? ({18'b0,A_m,6'b0}) : {48'b0} ;
        assign AB_m[7]= (B_m[7]==1'b1) ? ({17'b0,A_m,7'b0}) : {48'b0} ;
        assign AB_m[8]= (B_m[8]==1'b1) ? ({16'b0,A_m,8'b0}) : {48'b0} ;
```

```
assign AB_m[9]= (B_m[9]==1'b1) ? ({15'b0,A_m,9'b0}) : {48'b0} ;
assign AB_m[10]= (B_m[10]==1'b1) ? ({14'b0,A_m,10'b0}) : {48'b0
    } ;
assign AB_m[11]= (B_m[11]==1'b1) ? ({13'b0,A_m,11'b0}) : {48'b0
    } ;
assign AB_m[12]= (B_m[12]==1'b1) ? ({12'b0,A_m,12'b0}) : {48'b0
    } ;
assign AB_m[13]= (B_m[13]==1'b1) ? ({11'b0,A_m,13'b0}) : {48'b0
    } ;
assign AB_m[14]= (B_m[14]==1'b1) ? ({10'b0,A_m,14'b0}) : {48'b0
    } ;
assign AB_m[15]= (B_m[15]==1'b1) ? ({9'b0,A_m,15'b0})  : {48'b0
    } ;
assign AB_m[16]= (B_m[16]==1'b1) ? ({8'b0,A_m,16'b0})  : {48'b0
    } ;
assign AB_m[17]= (B_m[17]==1'b1) ? ({7'b0,A_m,17'b0})  : {48'b0
    } ;
assign AB_m[18]= (B_m[18]==1'b1) ? ({6'b0,A_m,18'b0})  : {48'b0
    } ;
assign AB_m[19]= (B_m[19]==1'b1) ? ({5'b0,A_m,19'b0})  : {48'b0
    } ;
assign AB_m[20]= (B_m[20]==1'b1) ? ({4'b0,A_m,20'b0})  : {48'b0
    } ;
assign AB_m[21]= (B_m[21]==1'b1) ? ({3'b0,A_m,21'b0})  : {48'b0
    } ;
assign AB_m[22]= (B_m[22]==1'b1) ? ({2'b0,A_m,22'b0})  : {48'b0
    } ;
assign AB_m[23]= (B_m[23]==1'b1) ? ({1'b0,A_m,23'b0})  : {48'b0
    } ;

// Tree based Adder Implementation

//Stage 1

customAdder U1(AB_m[0],AB_m[1],S_m[0]);
customAdder U2(AB_m[2],AB_m[3],S_m[1]);
customAdder U3(AB_m[4],AB_m[5],S_m[2]);
customAdder U4(AB_m[6],AB_m[7],S_m[3]);
customAdder U5(AB_m[8],AB_m[9],S_m[4]);
customAdder U6(AB_m[10],AB_m[11],S_m[5]);
customAdder U7(AB_m[12],AB_m[13],S_m[6]);
customAdder U8(AB_m[14],AB_m[15],S_m[7]);
customAdder U9(AB_m[16],AB_m[17],S_m[8]);
customAdder U10(AB_m[18],AB_m[19],S_m[9]);
customAdder U11(AB_m[20],AB_m[21],S_m[10]);
customAdder U12(AB_m[22],AB_m[23],S_m[11]);

//Stage 2

customAdder U13(S_m[0],S_m[1],Q_m[0]);
customAdder U14(S_m[2],S_m[3],Q_m[1]);
customAdder U15(S_m[4],S_m[5],Q_m[2]);
customAdder U16(S_m[6],S_m[7],Q_m[3]);
customAdder U17(S_m[8],S_m[9],Q_m[4]);
customAdder U18(S_m[10],S_m[11],Q_m[5]);

//Stage 3
```

```
        customAdder U19(Q_m[0],Q_m[1],Qn_m[0]);
        customAdder U20(Q_m[2],Q_m[3],Qn_m[1]);
        customAdder U21(Q_m[4],Q_m[5],Qn_m[2]);

        //Stage 4

        customAdder U22(Qn_m[0],Qn_m[1],U);
        customAdder U23(U,Qn_m[2],D);

endmodule


//Custom Adder Implementation

module customAdder(A_ca,B_ca,Sum_ca);

    input [47:0] A_ca;                                    //input  to
       Module
    input [47:0] B_ca;                                    //input  to
       Module
    output [47:0] Sum_ca;                                 //Output to
       Module

    wire [2:0] carry_temp;

   //Custom adder built using 3 BK adders

    BkAdder16 u1(A_ca[15:0],B_ca[15:0],1'b0,Sum_ca[15:0],carry_temp[0])
       ;
    BkAdder16 u2(A_ca[31:16],B_ca[31:16],carry_temp[0],Sum_ca[31:16],
       carry_temp[1]);
    BkAdder16 u3(A_ca[47:32],B_ca[47:32],carry_temp[1],Sum_ca[47:32],
       carry_temp[2]);


endmodule



//16 Bit BK adder Implementation


module BkAdder16(A_in,B_in,Carry_in,Sum_out,Carry_out);

    //Module port definitions
    input wire [15:0] A_in,B_in;
    input wire Carry_in;

    output wire [15:0]Sum_out;
    output wire Carry_out;

    //Intermediate wire connections
    wire [15:0] Prop,Gen;                    //Holds the Propogate and
       Generate signals from each stage
    wire[15:0] Carry_inter;                  //Holds the Carry_outs from
        each stage ie, Carry_inter[0]==> Carry_out of zeroth stage
    wire [14:0] Prop_inter;
```

```
wire [14:0] Gen_inter;


//Generation of P and G signals
PgGenerationBlock instance1(A_in,B_in,Prop,Gen);

//Generation of Group Generate and Propogate signals
//First level
GroupGPBlock instance1_0(Gen[1],Prop[1],Gen[0],Prop[0],Prop_inter
    [0],Gen_inter[0]);
GroupGPBlock instance3_2(Gen[3],Prop[3],Gen[2],Prop[2],Prop_inter
    [1],Gen_inter[1]);

GroupGPBlock instance5_4(Gen[5],Prop[5],Gen[4],Prop[4],Prop_inter
    [2],Gen_inter[2]);
GroupGPBlock instance7_6(Gen[7],Prop[7],Gen[6],Prop[6],Prop_inter
    [3],Gen_inter[3]);

GroupGPBlock instance9_8(Gen[9],Prop[9],Gen[8],Prop[8],Prop_inter
    [4],Gen_inter[4]);
GroupGPBlock instance11_10(Gen[11],Prop[11],Gen[10],Prop[10],
    Prop_inter[5],Gen_inter[5]);

GroupGPBlock instance13_12(Gen[13],Prop[13],Gen[12],Prop[12],
    Prop_inter[6],Gen_inter[6]);
GroupGPBlock instance15_14(Gen[15],Prop[15],Gen[14],Prop[14],
    Prop_inter[7],Gen_inter[7]);

//Second level
GroupGPBlock instance3_0(Gen_inter[1],Prop_inter[1],Gen_inter[0],
    Prop_inter[0],Prop_inter[8],Gen_inter[8]);
GroupGPBlock instance7_4(Gen_inter[3],Prop_inter[3],Gen_inter[2],
    Prop_inter[2],Prop_inter[9],Gen_inter[9]);
GroupGPBlock instance11_8(Gen_inter[5],Prop_inter[5],Gen_inter[4],
    Prop_inter[4],Prop_inter[10],Gen_inter[10]);
GroupGPBlock instance15_12(Gen_inter[7],Prop_inter[7],Gen_inter[6],
    Prop_inter[6],Prop_inter[11],Gen_inter[11]);

//Third level
GroupGPBlock instance7_0(Gen_inter[9],Prop_inter[9],Gen_inter[8],
    Prop_inter[8],Prop_inter[12],Gen_inter[12]);
GroupGPBlock instance15_8(Gen_inter[11],Prop_inter[11],Gen_inter
    [10],Prop_inter[10],Prop_inter[13],Gen_inter[13]);

//Final level
GroupGPBlock instance15_0(Gen_inter[13],Prop_inter[13],Gen_inter
    [12],Prop_inter[12],Prop_inter[14],Gen_inter[14]);

//Carry generation from group propogate and generate terms
assign Carry_inter[0]=Gen[0]|(Prop[0]&Carry_in);
assign Carry_inter[1]=Gen_inter[0]|(Prop_inter[0]&Carry_in);
assign Carry_inter[2]=Gen[2]|(Prop[2]&Carry_inter[1]);
assign Carry_inter[3]=Gen_inter[8]|(Prop_inter[8]&Carry_in);
assign Carry_inter[4]=Gen[4]|(Prop[4]&Carry_inter[3]);
assign Carry_inter[5]=Gen[5]|(Prop[5]&Carry_inter[4]);
assign Carry_inter[6]=Gen[6]|(Prop[6]&Carry_inter[5]);
assign Carry_inter[7]=Gen_inter[12]|(Prop_inter[12]&Carry_in);
assign Carry_inter[8]=Gen[8]|(Prop[8]&Carry_inter[7]);
```

```
    assign Carry_inter[9]=Gen[9]|(Prop[9]&Carry_inter[8]);
    assign Carry_inter[10]=Gen[10]|(Prop[10]&Carry_inter[9]);
    assign Carry_inter[11]=Gen[11]|(Prop[11]&Carry_inter[10]);
    assign Carry_inter[12]=Gen[12]|(Prop[12]&Carry_inter[11]);
    assign Carry_inter[13]=Gen[13]|(Prop[13]&Carry_inter[12]);
    assign Carry_inter[14]=Gen[14]|(Prop[14]&Carry_inter[13]);
    assign Carry_inter[15]=Gen_inter[14]|(Prop_inter[14]&Carry_in);

    //Carry Vector which holds the input carry to all stage 0 to 15
    wire [15:0] Carry_Vector={Carry_inter[14:0],Carry_in};

    //Final Carry Out
    assign Carry_out = Carry_inter[15];

    //Generating Sum at each stage by xor ing the propogate with
       Carryin of each stage
    assign Sum_out = Prop ^ Carry_Vector;

endmodule

//Group generate and propogate creation block
module GroupGPBlock(Gen_H,Prop_H,Gen_L,Prop_L,G_Prop,G_Gen);

    input wire Gen_H,Gen_L,Prop_H,Prop_L;
    output wire G_Gen,G_Prop;
    assign G_Gen = Gen_H |(Prop_H&Gen_L);
    assign G_Prop = Prop_H&Prop_L;

endmodule


//Propogate and Generate creation block
module PgGenerationBlock(A,B,P,G);

    input wire [15:0] A,B;
    output [15:0] P,G;

    assign P = A^B;
    assign G = A&B;


endmodule

//23 bit data MUX

module Mux(a,b,sel,y);

    input [22:0] a;
    input [22:0] b;
    input sel;
    output [22:0] y;
    assign y = (sel==1'b0)?a:b;

endmodule

//8 bit data MUX

module Mux8bit(a,b,sel,y);
```

```
    input [7:0] a;
    input [7:0] b;
    input sel;
    output [7:0] y;
    assign y = (sel==1'b0)?a:b;

endmodule
```

4.5.1 Parameterization and Modularity

The MAC unit is designed to be parameterized, allowing for easy adaptation to different data widths (e.g., 16-bit, 64-bit). This approach makes the design flexible and reusable across various applications. The modularity of the Verilog code ensures that each component (multiplier, adder, and accumulator) can be independently tested and modified without affecting other parts of the design.

4.6 Testbench Design

A functional testbench was created to verify the correctness of the MAC unit design. A testbench is a virtual environment in which the behavior of a digital system can be simulated and validated without the need for physical hardware. It allows us to apply stimulus (input values) to the design and observe the output to ensure the system works as expected.

4.6.1 Testbench Structure

The testbench is designed to simulate a variety of input combinations to test all possible edge cases and scenarios. It consists of:

- **Input Generation:** The testbench provides two 32-bit floating-point values as inputs to the MAC unit. These values represent the operands for the multiplication and accumulation operation.
- **Clock and Reset:** The clock signal drives the sequential operation of the MAC unit, while the reset signal initializes the system to a known state at the start of the simulation.
- **Output Monitoring:** The testbench continuously monitors the output of the MAC unit and checks that the results match expected values for each operation. Any mismatch is flagged for further investigation.

The Verilog code for the testbench is shown below:

Listing 2: Testbench for Floating-Point MAC Unit

```
`timescale 1ns / 1ps

module tb_mac;

    // Testbench Variables
    reg [31:0] In1, In2;       // Inputs to MAC unit
    reg CLK, rst;              // Clock and Reset
```

```
    wire [31:0] MacOut;        // Output of MAC unit

    // Instantiate the MAC Unit
    mac uut (
        .In1(In1),
        .In2(In2),
        .CLK(CLK),
        .rst(rst),
        .MacOut(MacOut)
    );

    // Clock Generation
    initial begin
        CLK = 0;
        forever #5 CLK = ~CLK; // Clock with a period of 10ns
    end

    // Test Vectors
    initial begin
        // Enable waveform dumping
        $dumpfile("mac_tb.vcd"); // Dump to "mac_tb.vcd" for GTKWave
        $dumpvars(0, tb_mac);    // Dump all variables in this
            testbench

        // Initialize Inputs
        In1 = 32'b0;
        In2 = 32'b0;
        rst = 0;

        //   reset
        #10;
        rst = 1;

        // Test Case 1: Multiply 1.5 * 2.5 and accumulate
        In1 = 32'b00111111110000000000000000000000; // 1.5 in IEEE 754
        In2 = 32'b01000000001000000000000000000000; // 2.5 in IEEE 754
        #10;
        $display("Test␣1:␣Time␣=␣%0t␣|␣In1␣=␣%h␣|␣In2␣=␣%h␣|␣MacOut␣=␣%
            h",
                 $time, In1, In2, MacOut);

        // Test Case 2: Multiply 0.5 * -4.0 and accumulate
        In1 = 32'b00111111000000000000000000000000; // 0.5 in IEEE 754
        In2 = 32'b11000000100000000000000000000000; // -4.0 in IEEE 754
        #10;
        $display("Test␣2:␣Time␣=␣%0t␣|␣In1␣=␣%h␣|␣In2␣=␣%h␣|␣MacOut␣=␣%
            h",
                 $time, In1, In2, MacOut);

        // Test Case 3: Multiply 10.0 * 0.1 and accumulate
        In1 = 32'b01000001001010000000000000000000; // 10.0 in IEEE 754
        In2 = 32'b00111101110011001100110011001101; // 0.1 in IEEE 754
        #10;
        $display("Test␣3:␣Time␣=␣%0t␣|␣In1␣=␣%h␣|␣In2␣=␣%h␣|␣MacOut␣=␣%
            h",
                 $time, In1, In2, MacOut);

        // Test Case 4: Reset the MAC Unit
```

```
        rst = 0;
        #10;
        rst = 1;
        $display("Test␣4:␣Time␣=␣%0t␣|␣Reset␣Applied␣|␣MacOut␣=␣%h",
                 $time, MacOut);

        // Test Case 5: Multiply 7.0 * 3.0 and accumulate
        In1 = 32'b01000000111000000000000000000000; // 7.0 in IEEE 754
        In2 = 32'b01000000010000000000000000000000; // 3.0 in IEEE 754
        #10;
        $display("Test␣5:␣Time␣=␣%0t␣|␣In1␣=␣%h␣|␣In2␣=␣%h␣|␣MacOut␣=␣%
           h",
                 $time, In1, In2, MacOut);

        // End Simulation
        $finish;
    end

    // Monitor Outputs
    initial begin
        $monitor("Time␣=␣%0t␣|␣In1␣=␣%h␣|␣In2␣=␣%h␣|␣MacOut␣=␣%h␣|␣rst␣
           =␣%b",
                 $time, In1, In2, MacOut, rst);
    end

endmodule
```

4.6.2 Edge Case Considerations

The testbench includes tests for various edge cases, such as:

- **Zero Input:** Verifies that the MAC unit correctly handles zero as an input operand.
- **Negative Numbers:** Ensures that the MAC unit produces the correct result when one or both operands are negative.
- **Large and Small Numbers:** Tests the unit's ability to handle numbers at the extremes of the floating-point range.
- **Overflow and Underflow:** Checks that the unit can handle overflow and underflow conditions by properly setting the exponent and mantissa fields.

4.7 Simulation and Verification

Simulation is a critical step in verifying the functionality of the MAC unit. The primary tool used for simulation is ModelSim or XSIM, both of which are capable of running Verilog testbenches and displaying waveforms. During the simulation:

- **Input Stimulus:** The testbench applies a series of inputs to the MAC unit, and the output is monitored.
- **Waveform Generation:** Simulation results are captured in waveform files, which show the changes in signal values over time. These waveforms are used to verify the correctness of the MAC operation.

- **Result Comparison:** The output from the MAC unit is compared to expected values to ensure that the unit performs correctly across all test cases.

4.8 Synthesis and Post-Synthesis Verification

Once the MAC unit is verified at the functional level, it is synthesized to generate the gate-level representation using synthesis tools like Synopsys Design Compiler or Cadence Genus. The synthesis process involves:

- **RTL to Gate Mapping:** The high-level Verilog code is mapped to specific logic gates, considering area, timing, and power constraints.

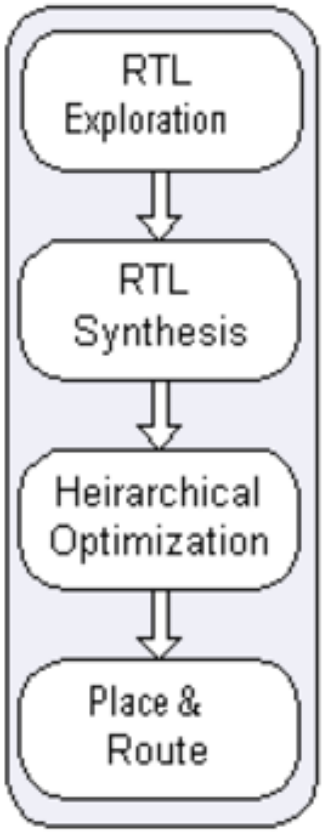

Figure 9: RTL Synthesis

- **Optimization:** The synthesis tool optimizes the design for power, area, and timing.
- **Post-Synthesis Simulation:** After synthesis, the gate-level netlist is used in post-synthesis simulations to ensure that the synthesized design behaves as expected.

5 Simulation Results

Simulation plays a vital role in verifying the functionality, performance, and correctness of digital designs before hardware implementation. For this project, simulations were performed at various stages—pre-synthesis, post-synthesis, and post-layout—to ensure the MAC unit meets the desired specifications. This section discusses the simulation setup,

methodology, and results for each stage of the design process, supported by graphical representations of the key metrics.

5.1 Pre-Synthesis Verification

Pre-synthesis verification is the first step in validating the functionality of the design. At this stage, the design exists as RTL (Register Transfer Level) code, typically written in Verilog or VHDL. The goal of pre-synthesis verification is to confirm that the MAC unit performs correctly according to its specification, even before the logic is mapped to actual hardware.

5.1.1 Testbenches and Input Generation

A testbench is a crucial component for pre-synthesis verification. It serves as the virtual environment in which the design is tested. The testbench for the MAC unit was written to apply various test vectors (input combinations) to the MAC unit and compare the output with expected values. The inputs consist of floating-point numbers, and the expected outputs are computed based on known mathematical operations, such as multiplication followed by accumulation.

- **Input Generation:** The testbench generates different combinations of 32-bit floating-point numbers to feed as inputs to the MAC unit. These inputs are selected to test the functionality under various conditions (e.g., zero, positive, negative, large, small numbers).
- **Clocking and Reset:** The testbench includes a clock generator to simulate sequential operation and a reset signal to initialize the design to a known state.
- **Expected Output:** The expected output is computed based on the given floating-point multiplication and accumulation operations, which are then compared to the actual output of the MAC unit during simulation.

5.1.2 Simulation Results and Waveforms

The simulation results for pre-synthesis verification were captured in waveform files. These waveforms display the signals and their transitions over time, allowing us to visualize how the MAC unit processes the inputs.

```
[EE671_44@vlsi73 verify_RTL]$ ./run-iverilog.sh
Testing: mac
VCD info: dumpfile mac_tb.vcd opened for output.
Time = 0 | In1 = 00000000 | In2 = 00000000 | MacOut = xxxxxxxx | rst = 0
Time = 7000 | In1 = 00000000 | In2 = 00000000 | MacOut = 00000000 | rst = 0
Time = 10000 | In1 = 3fc00000 | In2 = 40200000 | MacOut = 00000000 | rst = 1
Time = 17000 | In1 = 3fc00000 | In2 = 40200000 | MacOut = 00800000 | rst = 1
Test 1: Time = 20000 | In1 = 3fc00000 | In2 = 40200000 | MacOut = 00800000
Time = 20000 | In1 = 3f000000 | In2 = c0800000 | MacOut = 00800000 | rst = 1
Time = 27000 | In1 = 3f000000 | In2 = c0800000 | MacOut = 40700000 | rst = 1
Test 2: Time = 30000 | In1 = 3f000000 | In2 = c0800000 | MacOut = 40700000
Time = 30000 | In1 = 41280000 | In2 = 3dcccccd | MacOut = 40700000 | rst = 1
Time = 37000 | In1 = 41280000 | In2 = 3dcccccd | MacOut = 3fe00000 | rst = 1
Test 3: Time = 40000 | In1 = 41280000 | In2 = 3dcccccd | MacOut = 3fe00000
Time = 40000 | In1 = 41280000 | In2 = 3dcccccd | MacOut = 3fe00000 | rst = 0
Time = 47000 | In1 = 41280000 | In2 = 3dcccccd | MacOut = 00000000 | rst = 0
Test 4: Time = 50000 | Reset Applied | MacOut = 00000000
Time = 50000 | In1 = 40e00000 | In2 = 40400000 | MacOut = 00000000 | rst = 1
Time = 57000 | In1 = 40e00000 | In2 = 40400000 | MacOut = 00800000 | rst = 1
Test 5: Time = 60000 | In1 = 40e00000 | In2 = 40400000 | MacOut = 00800000
[EE671_44@vlsi73 verify_RTL]$
```

Figure 10: Pre-Synthesis Simulation Input Signals for MAC Unit

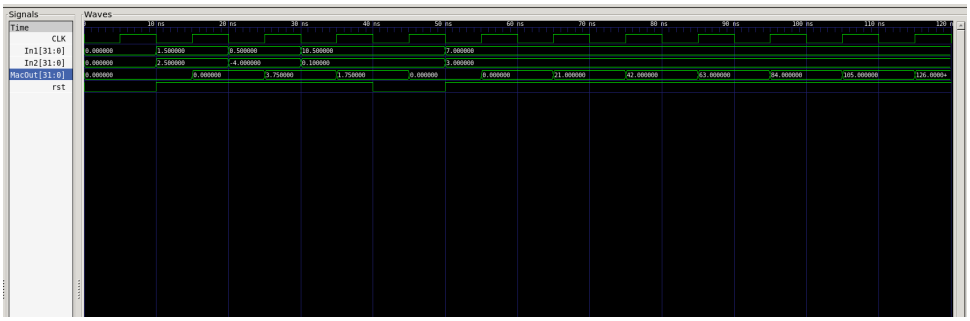

Figure 11: Pre-Synthesis Simulation Output Waveform for MAC Unit(Note : USE BitsToReal Data Format to visualise in GTKwave)

Figures 10 and 11 show the input signals (operands A and B) and the output of the MAC unit (MAC_OUT) at different clock cycles. These results confirm that the MAC unit correctly computes the product of A and B and adds it to the accumulator.

5.2 Post-Synthesis Verification

After the MAC unit is synthesized, the next step is post-synthesis verification. In this stage, the high-level RTL code is converted into a gate-level netlist, which is ready to be mapped to physical hardware. The primary objective of post-synthesis verification is to ensure that the synthesized design still performs correctly and that no errors were introduced during the synthesis process.

5.2.1 Simulation Results and Post-Synthesis Waveforms

Post-synthesis verification ensures that the synthesized design still adheres to the functionality of the original RTL. The same testbench used in pre-synthesis verification was applied to the synthesized design.

```
[EE671_44@vlsi73 verify_RTL]$ ./run-iverilog.sh
Testing: mac
VCD info: dumpfile mac_tb.vcd opened for output.
Time = 0 | In1 = 00000000 | In2 = 00000000 | MacOut = xxxxxxxx | rst = 0
Time = 7000 | In1 = 00000000 | In2 = 00000000 | MacOut = 00000000 | rst = 0
Time = 10000 | In1 = 3fc00000 | In2 = 40200000 | MacOut = 00000000 | rst = 1
Time = 17000 | In1 = 3fc00000 | In2 = 40200000 | MacOut = 00800000 | rst = 1
Test 1: Time = 20000 | In1 = 3fc00000 | In2 = 40200000 | MacOut = 00800000
Time = 20000 | In1 = 3f000000 | In2 = c0800000 | MacOut = 00800000 | rst = 1
Time = 27000 | In1 = 3f000000 | In2 = c0800000 | MacOut = 40700000 | rst = 1
Test 2: Time = 30000 | In1 = 3f000000 | In2 = c0800000 | MacOut = 40700000
Time = 30000 | In1 = 41280000 | In2 = 3dcccccd | MacOut = 40700000 | rst = 1
Time = 37000 | In1 = 41280000 | In2 = 3dcccccd | MacOut = 3fe00000 | rst = 1
Test 3: Time = 40000 | In1 = 41280000 | In2 = 3dcccccd | MacOut = 3fe00000
Time = 40000 | In1 = 41280000 | In2 = 3dcccccd | MacOut = 3fe00000 | rst = 0
Time = 47000 | In1 = 41280000 | In2 = 3dcccccd | MacOut = 00000000 | rst = 0
Test 4: Time = 50000 | Reset Applied | MacOut = 00000000
Time = 50000 | In1 = 40e00000 | In2 = 40400000 | MacOut = 00000000 | rst = 1
Time = 57000 | In1 = 40e00000 | In2 = 40400000 | MacOut = 00800000 | rst = 1
Test 5: Time = 60000 | In1 = 40e00000 | In2 = 40400000 | MacOut = 00800000
[EE671_44@vlsi73 verify_RTL]$
```

Figure 12: Post-Synthesis Simulation Input Signals for MAC Unit

Figure 13: Post-Synthesis Simulation Output Waveform for MAC Unit(Note : USE BitsTo-Real Data Format to visualise in GTKwave)

Figures 12 and 13 illustrate the post-synthesis simulation results. The outputs match the pre-synthesis results, confirming that the functionality has been preserved during synthesis.

5.3 Performance Metrics

To evaluate the efficiency and practicality of the MAC unit, several key metrics were analyzed during synthesis and layout stages.

5.3.1 Standard Cell Usage

The number of standard cells used in the design reflects its area efficiency. A snapshot of the standard cell count is shown in Figure 14.

```
[EE671_44@vlsi73 synthesis]$ cat mac.v|grep "sky130_fd_sc_hd__" | wc -l
4447
```

Figure 14: Number of Standard Cells Used in the MAC Unit

5.3.2 Power Consumption

The power consumption of the MAC unit was measured during post-layout simulations. Figure 15 provides a breakdown of the total power consumed by the sequential, combinational, and clocking components.

```
 report_power
============================================================================
======================== Typical Corner ====================================

Group                  Internal  Switching    Leakage      Total
                          Power      Power      Power      Power (Watts)
----------------------------------------------------------------------------
Sequential             1.95e-04   2.93e-05   8.08e-10   2.24e-04   8.3%
Combinational          1.33e-03   1.05e-03   1.45e-08   2.38e-03  87.6%
Clock                  7.02e-05   4.26e-05   8.51e-10   1.13e-04   4.2%
Macro                  0.00e+00   0.00e+00   0.00e+00   0.00e+00   0.0%
Pad                    0.00e+00   0.00e+00   0.00e+00   0.00e+00   0.0%
----------------------------------------------------------------------------
Total                  1.60e-03   1.12e-03   1.62e-08   2.71e-03 100.0%
                          58.8%      41.2%       0.0%
```

Figure 15: Power Consumption Breakdown for the MAC Unit

5.3.3 Chip Area

The total area occupied by the MAC unit was determined during the layout process. Figure 16 shows the final layout view highlighting the physical area used by the design.

```
Chip area for module '\mac': 40024.636800
```

Figure 16: Chip Area Utilized by the MAC Unit

5.3.4 Skew and Slack Analysis

Skew and slack are critical timing parameters that influence the performance and reliability of digital designs.

- **Clock Skew:** Clock skew refers to the difference in the arrival times of the clock signal at different points within the design. Excessive skew can lead to setup or hold time violations, causing incorrect functionality.
- **Timing Slack:** Timing slack represents the difference between the required time and the actual time taken for a signal to propagate through a path. Positive slack indicates that the design meets timing requirements, while negative slack implies violations.

In this project, the MAC unit's skew and slack were analyzed during post-layout simulations to ensure the design adheres to timing constraints. Figure 17 illustrates the results of the skew and slack analysis.

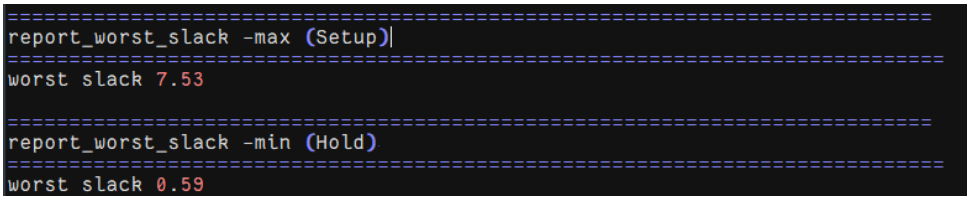

Figure 17: Skew and Slack Analysis for the MAC Unit

6 Physical Design

The physical design process involves transforming the synthesized netlist into a layout suitable for fabrication. This process includes several stages: floorplanning, placement, clock tree synthesis (CTS), routing, and final layout verification. Each of these stages is crucial in ensuring the overall performance, manufacturability, and functionality of the MAC unit. The following sections provide a detailed explanation of each step, along with the results and insights from the design flow.

6.1 Floorplan

Floorplanning is the first and one of the most critical steps in physical design. It involves defining the die area and strategically placing macros, standard cells, and routing resources. A well-designed floorplan optimizes the use of area, reduces signal delay, and ensures efficient power distribution.

6.1.1 Objective

The primary goals of floorplanning are to:

- Allocate sufficient space for each module (e.g., multiplier, adder, accumulator).
- Optimize the arrangement of components to minimize routing congestion and delay.
- Plan the power distribution network, including power rings and straps.
- Ensure that the design fits within the defined die area, leaving adequate space for routing and power delivery.

6.1.2 Process

The floorplanning was executed using the `run_floorplan` command in OpenLane, which generates a DEF (Design Exchange Format) file. This file defines the layout of the design and allocates space for the cells. We used Magic, a layout visualization tool, to view and verify the floorplan.

Key considerations during floorplanning include:

- Core Area: The area allocated for the functional blocks of the MAC unit.
- Aspect Ratio: The design was kept close to a square shape to minimize routing wire length.
- Power Planning: Power grids were defined to ensure efficient power delivery to all cells.

6.1.3 Results

The floorplan of the MAC unit is shown in Figure 18. It highlights the placement of major modules and power grids, confirming that the space is efficiently utilized.

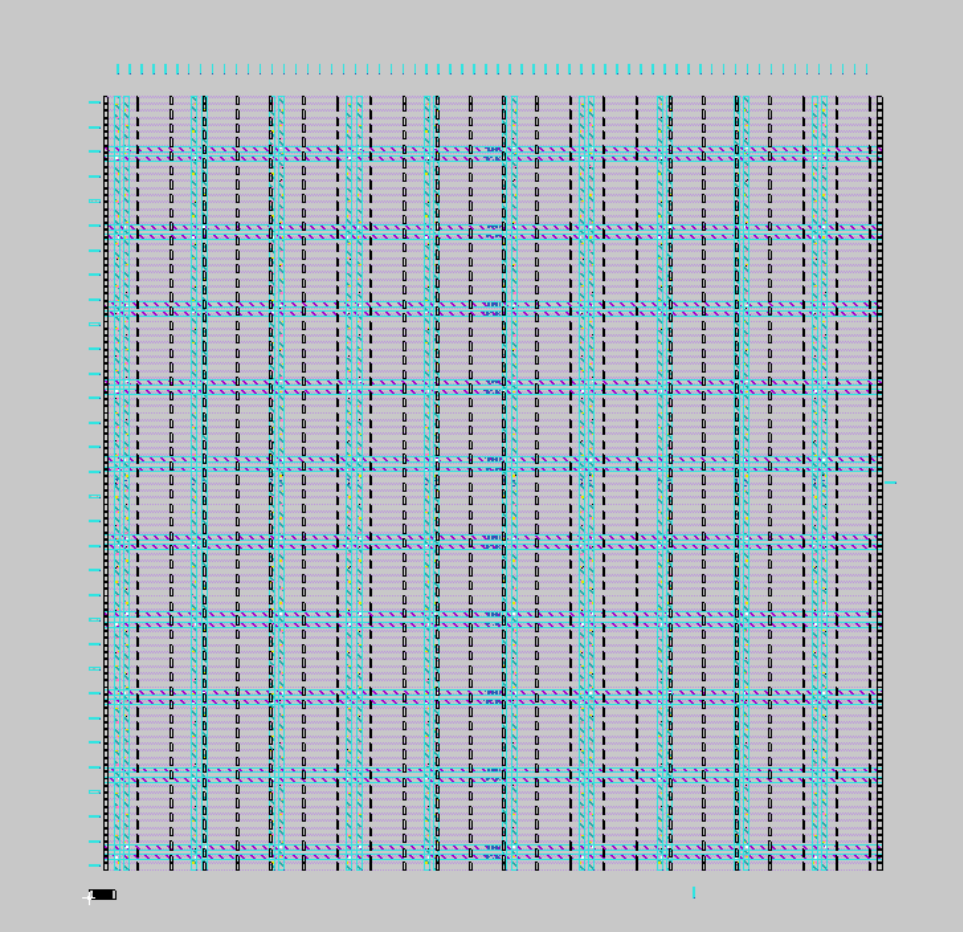

Figure 18: Floorplan of the MAC unit

6.2 Placement

Placement arranges the standard cells (generated during synthesis) and macros within the core area defined in the floorplan. The goal of placement is to minimize interconnect lengths, reduce signal delay, and optimize the overall design for performance and area.

6.2.1 Objective

The key objectives of placement include:

- Minimize wire lengths to reduce delay and power consumption.
- Ensure cells are placed within the allocated regions from the floorplan.
- Meet timing constraints by considering the interconnect delays between critical components.

6.2.2 Process

Placement was done using the `run_placement` command. After placement, the DEF file was reviewed in Magic to ensure no cell overlaps, and to check for any misplacement or congestion issues. The placement algorithm optimizes the cell positions while maintaining the legal constraints.

The placement process includes:

- Legalization: Ensuring that no cells overlap or violate design rules.
- Congestion Analysis: Identifying and resolving areas of high cell density to avoid routing congestion.
- Timing Optimization: Adjusting the placement to improve timing by reducing critical path delays.

6.2.3 Results

Figure 19 shows the placement of the MAC unit. The design is well-spread, with enough space for routing, and no placement violations were observed.

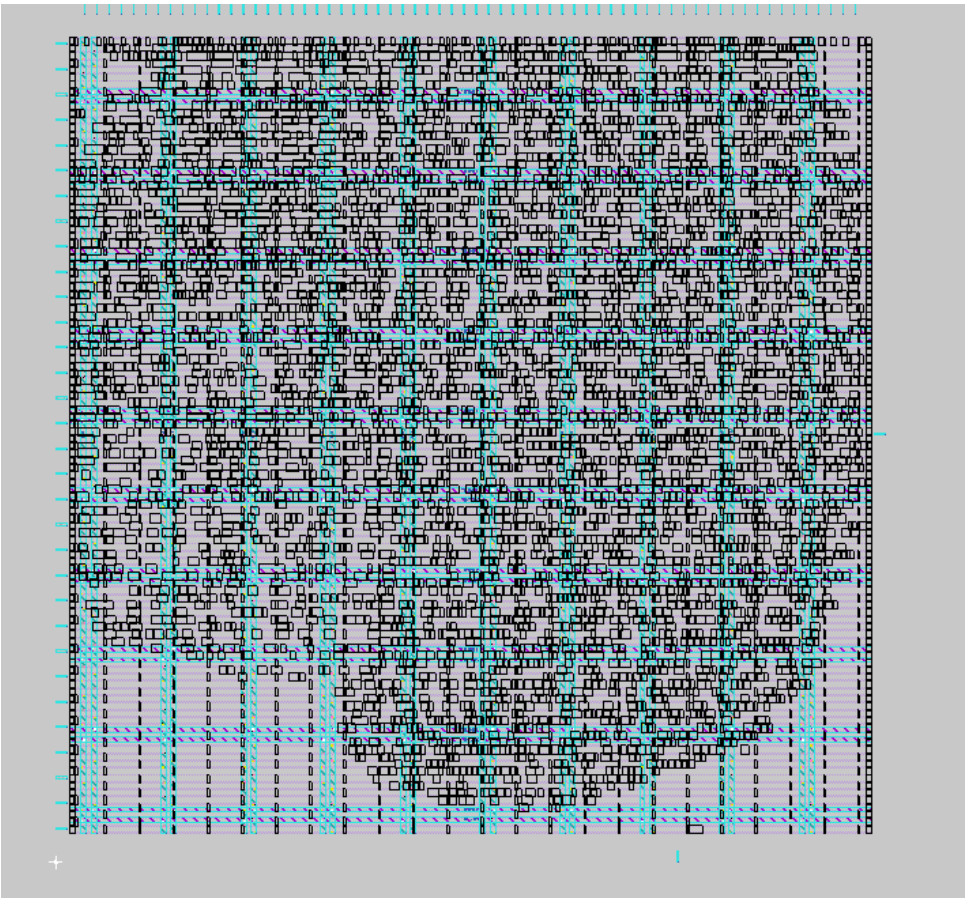

Figure 19: Placed design of the MAC unit

6.3 Clock Tree Synthesis (CTS)

Clock Tree Synthesis (CTS) is an essential step in ensuring that the clock signal is delivered to all sequential elements (flip-flops, registers) in the design with minimal skew and

timing violations.

6.3.1 Objective

The primary objectives of CTS include:

- Minimize clock skew to ensure all sequential elements receive the clock signal at the correct time.
- Insert buffers and inverters as necessary to balance the clock tree and reduce delay.
- Optimize power consumption by limiting the number of buffers and minimizing the clock network's area.

6.3.2 Process

CTS was performed using the `run_cts` command. After clock tree synthesis, the DEF file was opened in Magic to ensure proper clock routing. The clock network was analyzed for skew and balanced distribution.

The process involves:

- Buffer Insertion: Adding buffers to balance the clock distribution and minimize skew.
- Clock Path Optimization: Ensuring that the clock signal reaches all flip-flops within the timing constraints.
- Timing Analysis: Verifying that the clock paths meet setup and hold timing constraints.

6.3.3 Results

Figure 20 illustrates the design after CTS, showing the balanced clock tree and the distribution of clock buffers.

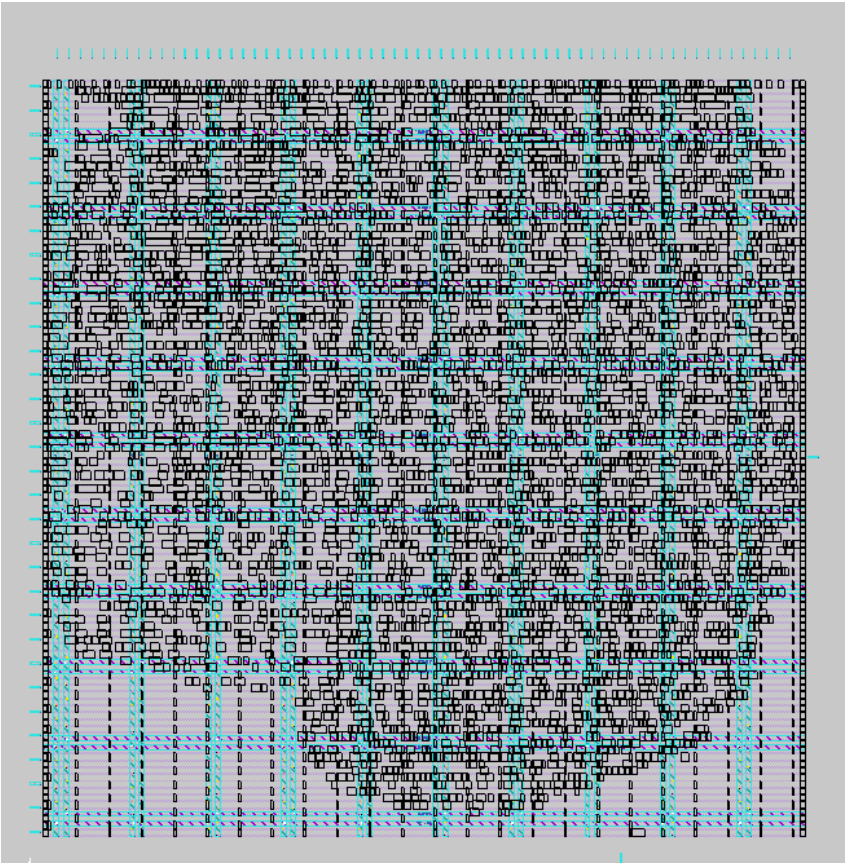

Figure 20: Design after Clock Tree Synthesis (CTS)

6.4 Routing

Routing connects the placed cells and establishes physical paths for signals. This step is critical in ensuring that all signals are correctly routed without violating design rules.

6.4.1 Objective

The main objectives of routing are:

- Connect all cells and macros while adhering to design rules (e.g., minimum width, spacing).
- Minimize wirelength to reduce delay and power consumption.
- Avoid congestion and ensure that critical signals are routed efficiently.

6.4.2 Process

Routing was completed using the `run_routing` command. The routing process was performed in two stages:

- Global Routing: Establishing rough routes for all nets.
- Detailed Routing: Assigning specific tracks to the global routes and ensuring DRC compliance.

The design was then checked for any violations (e.g., shorts, opens) using the Design Rule Check (DRC) tool.

6.4.3 Results

The routed design of the MAC unit is shown in Figure 21. The figure demonstrates that all nets were successfully routed without any violations.

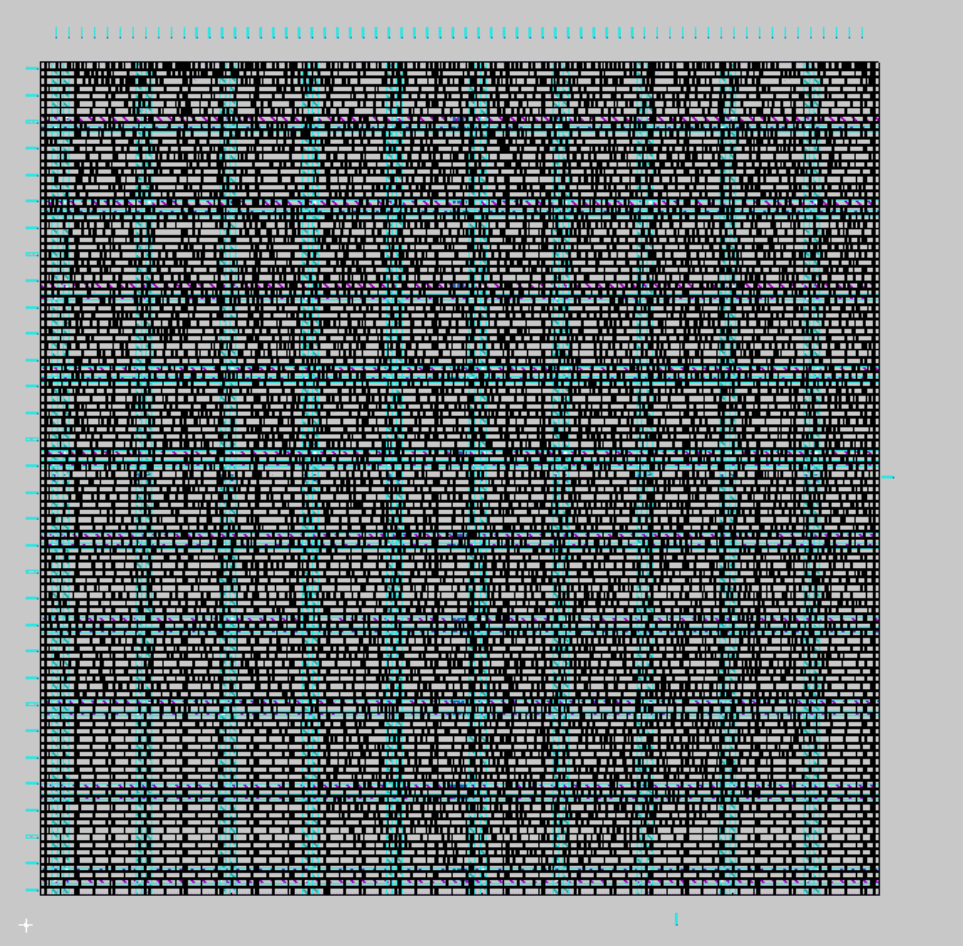

Figure 21: Routed design of the MAC unit

6.5 Final Layout (GDS)

The final layout represents the complete chip design, including all standard cells, macros, power distribution, and routing. This layout is used for generating the GDS (Graphic Data System) file, which is sent for fabrication.

6.5.1 Results

The final GDS layout was visualized in Magic to ensure that all components and routing layers were correctly placed. The expanded view of the layout is shown in Figure 22.

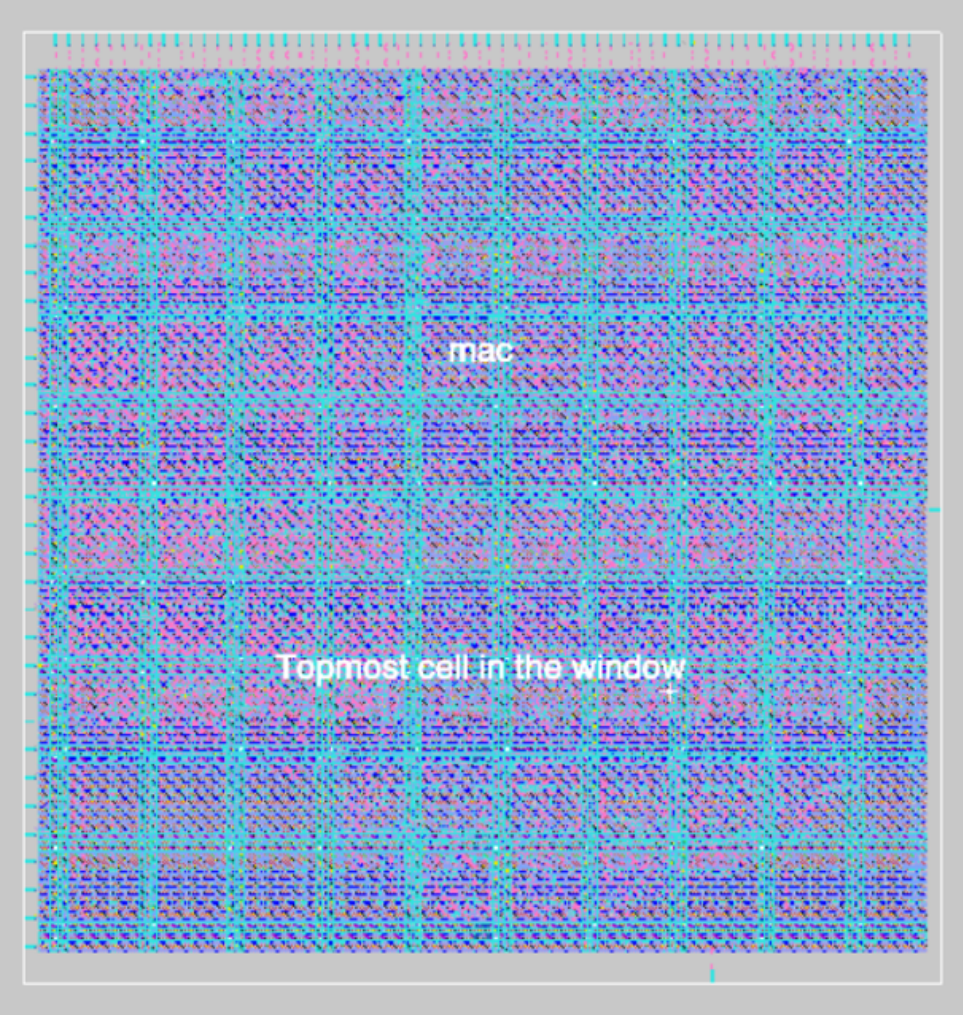

Figure 22: Final layout of the MAC unit

6.6 Post Physical Design RTL Verification

After the physical design process was completed, the post-physical design RTL was verified to ensure that the final netlist, post-placement and routing, functions as expected.

6.6.1 Objective

The primary objective of post-physical design RTL verification is to ensure that the functionality of the design, as described by the RTL, is maintained after the physical design process. This is done by simulating the netlist after placement and routing and comparing the output with the expected values from the functional specification.

6.6.2 Methodology

The post-physical design RTL verification was performed using the same testbench used in previous RTL simulations. The simulation was run, and the resulting waveform was observed using GTKWAVE to verify the correctness of the design.

The following steps were followed:

- Testbench: The same functional testbench used for pre-synthesis and post-synthesis verification was used to simulate the post-physical design RTL.
- Waveform Generation: The simulation produces waveform data, which is captured in a GTKWAVE dump, allowing detailed inspection of the signal transitions.
- Signal Comparison: The output signals are compared against the expected values to verify that the design's functional behavior is correct after the physical design steps.

6.6.3 Results

The GTKWAVE waveform dump from the simulation of the post-physical design RTL is shown in Figure 23. The waveform illustrates the computation of the MAC unit, confirming that the multiplication and accumulation operations are functioning as expected. All input signals are processed correctly, and the output matches the expected behavior.

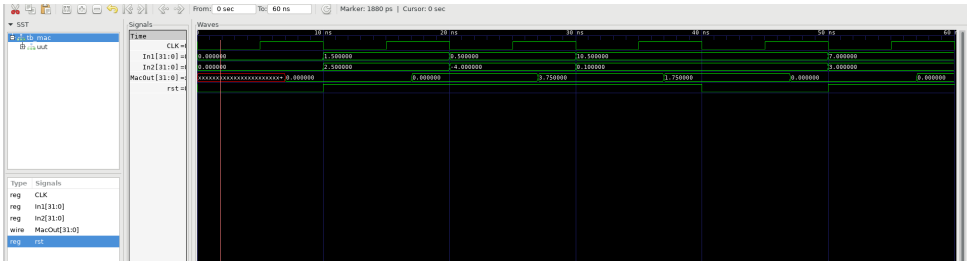

Figure 23: Functional simulation of the post-physical design RTL

6.7 LVS Report

The Layout Versus Schematic (LVS) check was performed to ensure that the layout matches the schematic and no errors were introduced during physical design. The `lvs.rpt` file was generated and analyzed to confirm the correctness of the layout.

6.7.1 Results

The LVS report shows that there are no discrepancies between the schematic and the layout. A screenshot of the LVS report with 0 errors is shown in Figure 24.

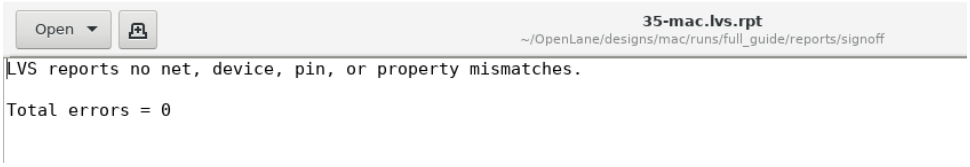

Figure 24: LVS report showing 0 errors

6.8 DRC Report

The Design Rule Check (DRC) was conducted to ensure that the layout adheres to manufacturing rules. The `drc.rpt` file confirmed that the design complies with all DRC rules.

6.8.1 Results

A screenshot of the DRC report showing 0 errors is shown in Figure 25.

Figure 25: DRC report showing 0 errors

6.9 STA Report and Power Consumption

The Static Timing Analysis (STA) and power consumption were evaluated to ensure that the MAC unit meets the required performance and power constraints. The `grt_sta.log` file was analyzed for timing data, and power consumption was extracted from the power report.

6.9.1 Results

The timing and power metrics extracted from the log file are summarized in Table 1.

Metric	Value
Clock Frequency (MHz)	40
No. of standard cells	4447
Worst Case Setup Slack (ns)	7.53
Worst Case Hold Slack (ns)	0.59
Design Area (μm^2)	35130
Power Consumption (Sequential) (mW)	0.224
Power Consumption (Combinational) (mW)	2.38
Power Consumption (Clock) (mW)	0.113
Total Power Consumption (mW)	2.71

Table 1: Summary of STA and power consumption results for the MAC unit

7 Group Work Division

The work was divided as follows:

- **Sarim Zafeer Farooqui and Sivaganesh Locharla:**
  1. **Adder Block Design:**
     - Designed and implemented an efficient adder block for the MAC unit, ensuring accurate and high-speed addition of intermediate results.
     - Focused on optimizing the critical path delay and minimizing area and power consumption while maintaining reliability.

2. **Testbench Development:**
   - Tested the functionality of the adder with a testbench, ensuring all different cases were tested and verified.
   - Optimized the testbench for the MAC unit to test and verify various input cases effectively.
3. **Code Synthesis Optimization:**
   - Modified the Verilog code to ensure synthesizability by adhering to hardware design constraints.
   - Replaced non-synthesizable constructs with hardware-compatible equivalents.
   - Verified the compatibility of the code with synthesis tools, resolving warnings related to combinational feedback and uninitialized signals.
   - Ensured that the code adheres to best practices for clock domain management and sequential logic implementation.
4. **Physical Design Tasks:**
   - Ran simulations and resolved errors during routing stages, performing the following steps:
     (a) `run_synthesis`
     (b) `run_floorplan`
     (c) `run_placement`
     (d) `run_cts`
     (e) `run_routing`
     (f) `run_parasitics_sta`
     (g) `run_magic`
     (h) `run_magic_spice_export`
     (i) `run_magic_drc`
     (j) `run_lvs`
     (k) `run_antenna_check`
   - Extracted all log files after post-CTS and post-routing stages.
   - Read all DEF files from floorplan, placement, CTS, routing, and the final GDS file.
5. **Physical Verification:**
   - Verified the design by ensuring zero DRC errors and successful LVS checks.
   - Extracted log files for DRC and antenna checks to document compliance.
6. **Report Writing:**
   - Compiled and structured the project report, ensuring all tasks, results, and methodologies were presented comprehensively.
   - Organized and formatted sections, including physical design steps, verification results, and overall contributions of the team.

Gokul L P and Muhammad Anas A K:

1. **Architectural Design:**

- Designed the architecture of the floating-point multiplier, ensuring compatibility with the MAC unit's requirements.

2. **RTL Coding and Verification:**

   - Implemented RTL coding for the floating-point multiplier.
   - Developed a robust testbench to verify the functionality of the floating-point multiplier under various conditions.

3. **Integration Tasks:**

   - Integrated the multiplier and adder units to complete the MAC unit.
   - Developed a zero-detect block to optimize the performance of the MAC unit by minimizing unnecessary computations.

4. **Simulation and Testing:**

   - Simulated and tested the MAC unit to ensure correct functionality and performance.
   - Resolved synthesis issues encountered during the design process.

5. **Synthesis and Post-Synthesis Simulation:**

   - Successfully synthesized the complete MAC unit.
   - Performed post-synthesis simulations to verify the design's correctness and timing compliance.

8 Conclusion

The floating-point MAC unit was successfully designed, verified, and implemented. Its optimized performance metrics make it suitable for ANN accelerators. Future work includes enhancing power efficiency and integrating the design into larger systems.